

TWO FRAMEWORKS, TWO PHILOSOPHIES

Before diving into LlamaIndex, it helps to understand the fundamental design question every LLM framework must answer: *what is the primary abstraction?*

Two dominant philosophies have emerged:

- Chain-Centric Design (LangChain's Approach)
- Data-Centric Design (LlamaIndex's Approach)

Chain-Centric Design
(LangChain's Approach)



LangChain

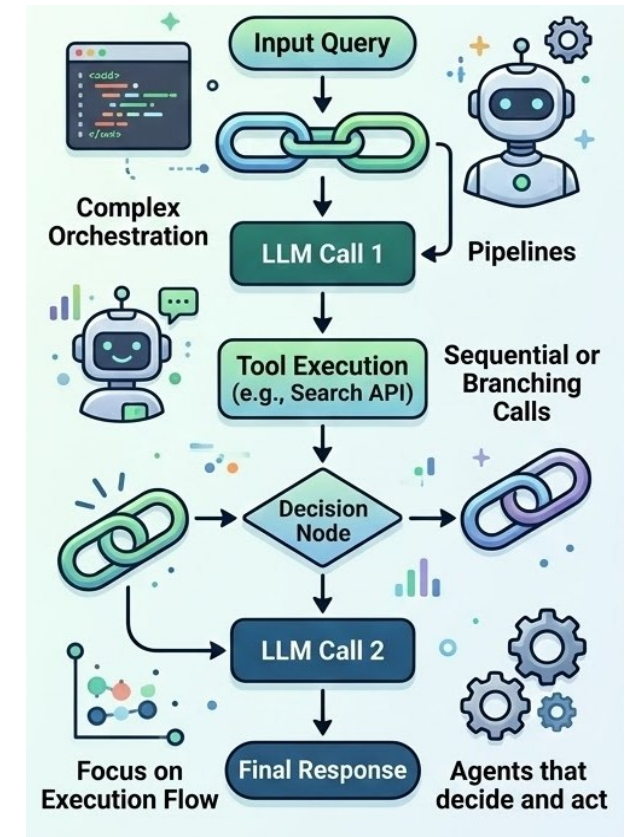
Data-Centric Design
(LlamaIndex's Approach)



LlamaIndex

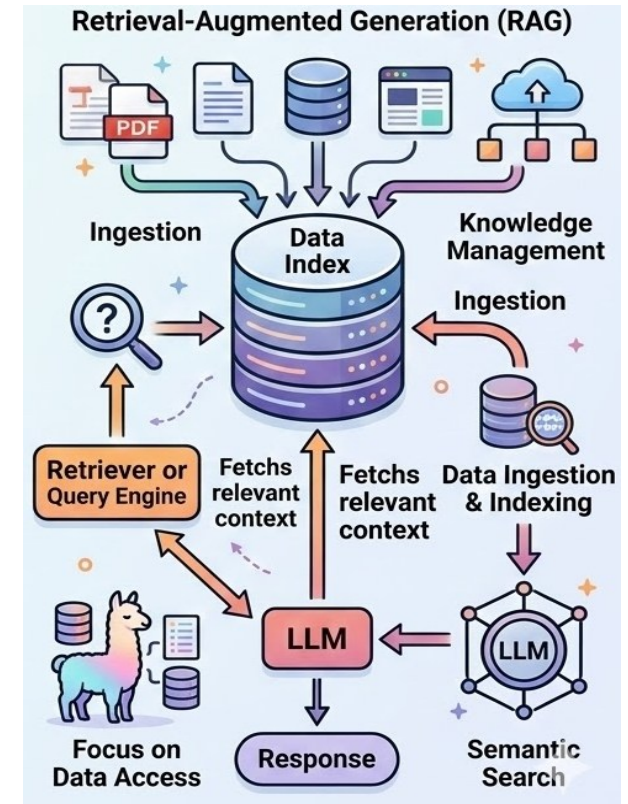
CHAIN-CENTRIC

- LangChain was built around the idea that LLM applications are sequences of steps, or chains
- The core abstraction is: *connect components together in a pipeline, where the output of one step feeds into the next*
- In LangChain, we think in terms of:
 - **Runnables:** components that can be invoked; form a part of the flow
 - **Chains:** linear sequences of runnables and other components
 - **Memory:** conversation state carried across chain invocations
 - **Data & Retrieval:** components specialised to serve storage and retrieval purposes
- This is a general-purpose design; LangChain can build chatbots, agents, pipelines, workflow automations: anything that chains LLM calls together
- RAG is one of many things it can do, but it is not the thing the framework was designed around



DATA-CENTRIC

- LlamaIndex was built around a different question:
how do I connect an LLM to my data?
- The core abstraction is not a chain, but a structured representation of your data that an LLM can query
- In LlamaIndex, we think in terms of:
 - **Data connectors:** loaders that ingest data from various sources
 - **Documents and Nodes:** data structured for retrieval
 - **Indices:** how your data is organised for search
 - **Query engines:** the interface that retrieves and generates answers
- This is a data-first, RAG-optimised design
- Everything in LlamaIndex is built to make the path from raw data to grounded LLM answers as smooth as possible



WHY LLAMA INDEX

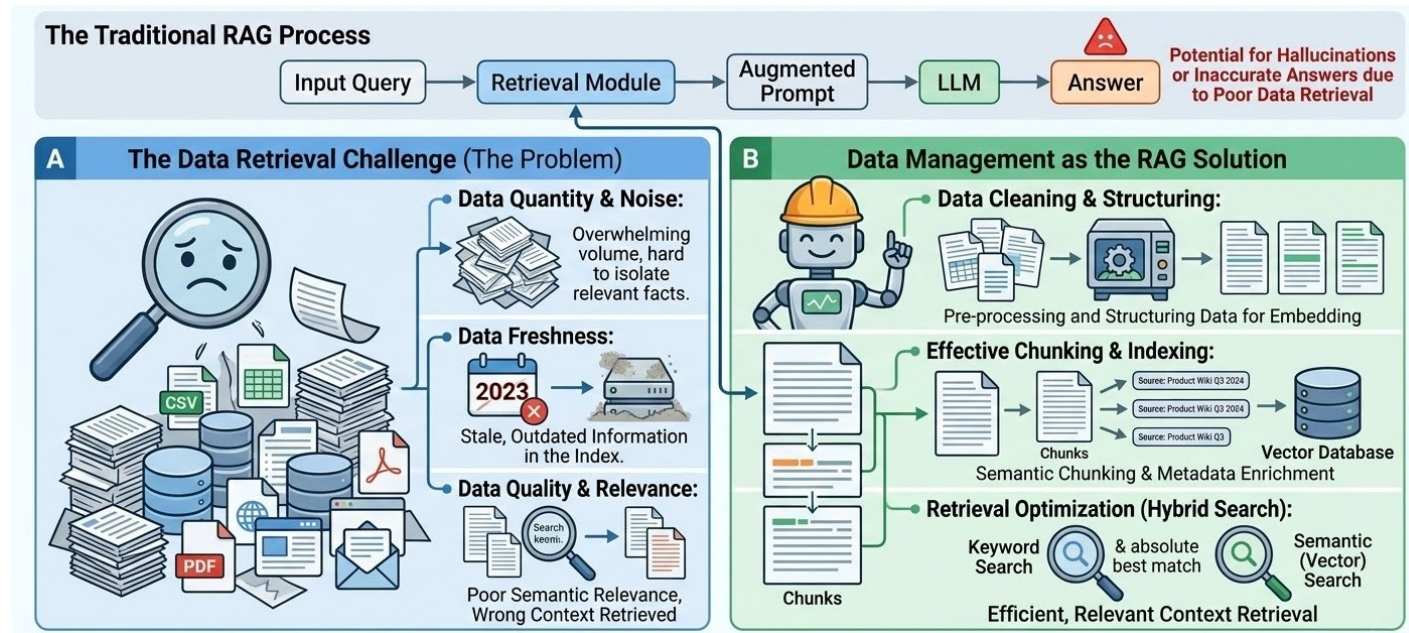
- RAG is fundamentally a data problem
- Consider what a RAG system actually does:
 1. **Ingest** data from heterogeneous sources (PDFs, databases, APIs, web pages)
 2. **Parse** and clean that data into a uniform representation
 3. **Chunk** it into retrieval-friendly pieces
 4. **Embed** and **index** those pieces for fast similarity search
 5. **Retrieve** relevant pieces at query time
 6. **Synthesise** an LLM response grounded in the retrieved context

Steps 1-4 are entirely about **data engineering**; Step 5 is **information retrieval**
Only step 6 involves an LLM call

A chain-centric framework will treat all of this as “steps before the chain runs”
A data-centric framework will treat it as core workflow

WHY LLAMAINDEX

- LlamaIndex provides dedicated, opinionated abstractions for every one of these steps
- We do not assemble a RAG pipeline from generic building blocks
- We use purpose-built components that know about documents, chunks, embeddings, and retrieval natively

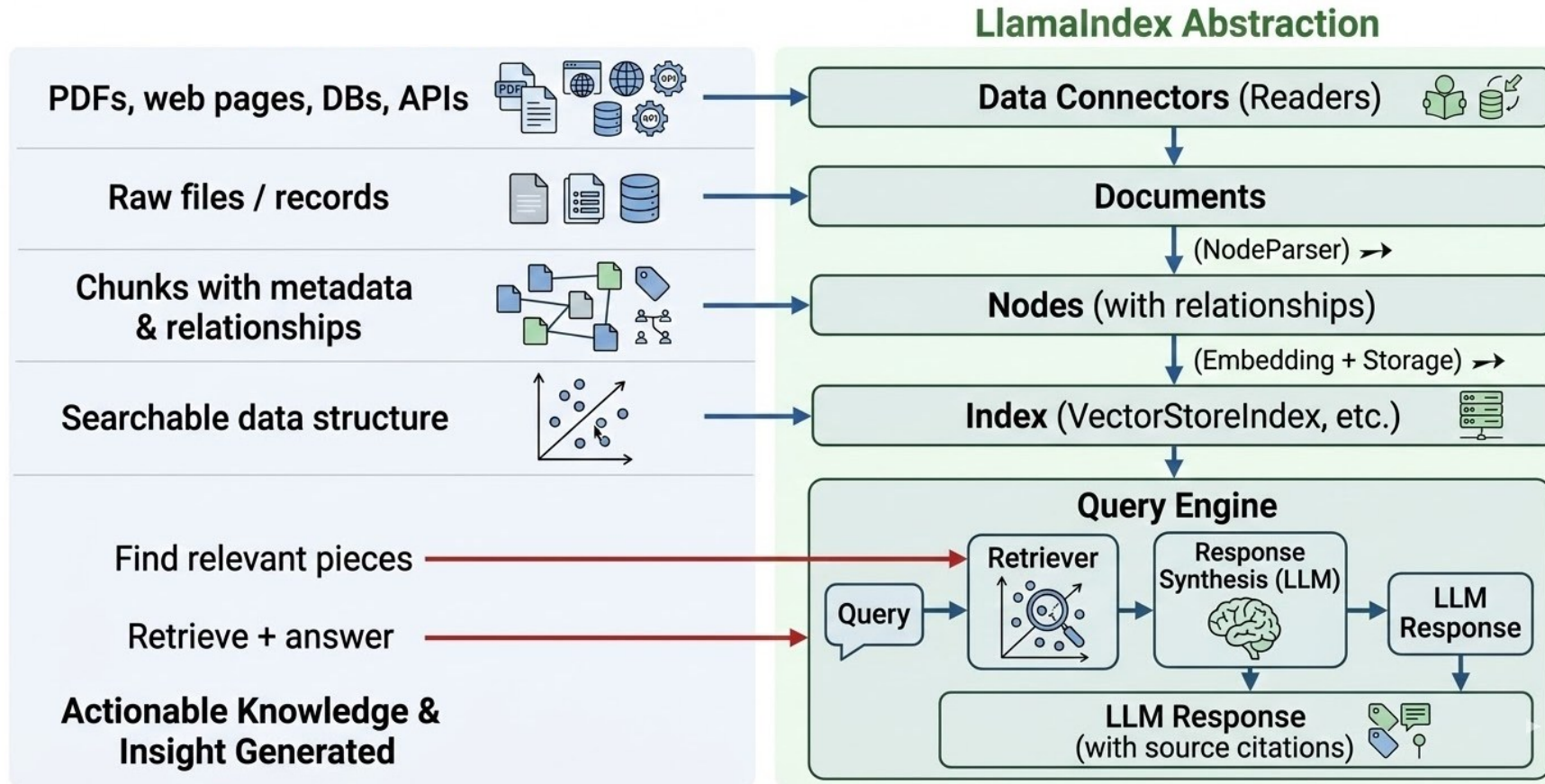


WHERE LANGCHAIN WINS

- LlamaIndex's data focus is a strength for RAG but a constraint for other patterns
- LangChain is the better choice when:
 - We need **complex agent workflows** with tool calling, planning, and dynamic routing
 - We are building a **general-purpose chatbot** that does many things beyond document Q&A
 - We need fine-grained control over every **link in the chain** (custom prompt templates, custom parsers, custom output handlers)
 - The application is **orchestration-heavy**: coordinating multiple LLM calls, APIs, and external services
 - We want to use LangGraph for **stateful, multi-step** agent architectures
- Many production systems use both: LlamaIndex for the data/retrieval layer, and LangChain (or LangGraph) for the orchestration/agent layer

LlamaIndex's Core Abstractions

LLAMAINDEX COMPONENTS



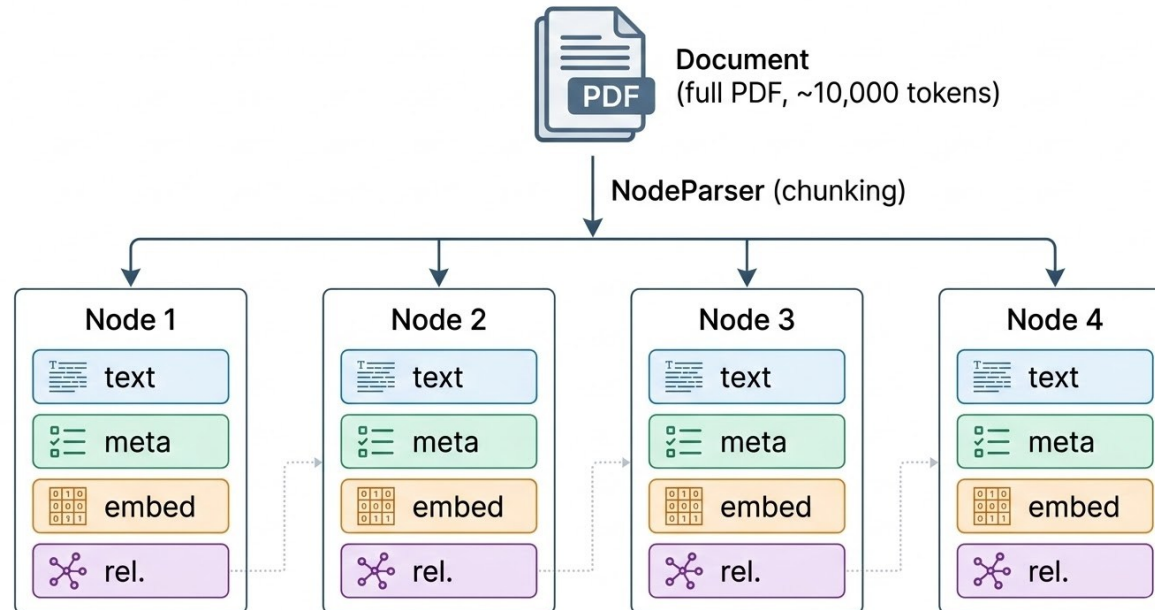
CORPUS → DOCUMENTS

- A **Document** in LlamaIndex is the raw unit of data ingestion
 - **text**: the raw content
 - **metadata**: a dictionary of structured attributes (filename, page number, source, custom fields)
 - **doc_id**: a unique identifier
- It represents a single source item - a PDF file, a web page, a database row, a Notion page, a Slack thread, etc.
- **Data connectors** (also called **Readers**) handle the translation from external sources to Document objects
- LlamaIndex ships with 160+ connectors via [LlamaHub](#), covering:
 - **Files**: PDF, DOCX, PPTX, Markdown, CSV, JSON, HTML, EPUB
 - **Databases**: PostgreSQL, MySQL, MongoDB, SQLite
 - **Cloud storage**: S3, Google Drive, Dropbox, OneDrive
 - **SaaS tools**: Notion, Slack, Confluence, Jira, Discord, GitHub
 - **Web**: Web pages, sitemaps, RSS feeds
 - **Specialised**: YouTube transcripts, arXiv papers, Wikipedia

CHUNKS → NODES

A **Node** is LlamaIndex's term for a chunk or a smaller piece of a Document that becomes a unit of retrieval

The Document → Node transformation is where chunking happens



MORE ON NODES

Nodes are more than just text chunks; Each node carries:

Field	Description
<i>text</i>	The chunk content
<i>metadata</i>	Inherited from the parent Document (plus any node-level additions)
<i>embedding</i>	The vector representation (computed during indexing)
<i>relationships</i>	Links to other nodes — parent Document, previous/next nodes, child nodes

Unlike flat chunks in other frameworks, LlamaIndex nodes are **connected**

Each node knows:

- Its **parent** Document
- Its **previous** and **next** sibling nodes (adjacent chunks from the same document)
- Optionally, **child** nodes (if hierarchical parsing is used)

NODE RETRIEVAL PATTERNS

○ Context window expansion

- Retrieve Node 3, but also pull in Node 2 and Node 4 for surrounding context
- *Use case:* Ideal for legal or technical documentation where a single relevant paragraph (Node 3) references "the aforementioned clause" or "the previous step," necessitating the retrieval of surrounding blocks (Node 2 & 4) to resolve those dependencies

○ Parent retrieval

- Retrieve at the node level for precision, but return the full parent Document for completeness
- *Use case:* Perfect for long-form research papers where you want to find the exact table or figure caption for high-accuracy retrieval (node level) but need to provide the model with the entire section (parent document) to ensure it understands the methodology and results

○ Hierarchical retrieval

- Index at multiple granularities (sentence-level and paragraph-level) and route queries to the appropriate level
- *Use Case:* Useful for complex knowledge bases where a user might ask a granular question ("What is the torque spec for Bolt X?") requiring sentence-level retrieval, or a broad question ("Give me an overview of the engine assembly") requiring higher paragraph-level or summary-level retrieval

VECTOR STORE → INDEX

An **Index** is the data structure that organises nodes for efficient retrieval – it is the bridge between raw data (nodes) and query-time search

LlamaIndex supports several index types:

Index	How It Works	Best For
<i>Common Approaches</i>		
<i>VectorStoreIndex</i>	Embeds all nodes and stores in a vector store; retrieval via ANN similarity search	Standard semantic search (most common)
<i>SummaryIndex</i> (formerly <i>ListIndex</i>)	Stores all nodes in a flat list; at query time, summarises sequentially or in parallel	When you need the LLM to read all nodes (full-corpus summarisation)
<i>TreeIndex</i>	Builds a hierarchical tree of summaries; queries traverse from root to leaves	Hierarchical summarisation over large corpora
<i>Advanced Approaches</i>		
<i>KeywordTableIndex</i>	Extracts keywords from each node; retrieval via keyword matching	When keyword/entity matching is important
<i>KnowledgeGraphIndex</i>	Builds a knowledge graph from extracted triplets	Relationship-heavy domains

RETRIEVER → QUERY ENGINE

This is where retrieval meets generation. LlamaIndex separates these into two components:

1. Retriever

- A **Retriever** is responsible for fetching relevant nodes from the index given a query
- It handles *only* retrieval and no LLM call
- We can customise retrieval behaviour:
 - *similarity_top_k*: number of nodes to retrieve
 - *filters*: metadata filters (equivalent to ChromaDB's where clause)
 - custom retrievers that implement our own retrieval logic (hybrid, recursive, auto-merging)

2. Query Engine

- A **Query Engine** wraps a **retriever** and a **response synthesiser**
- It handles the full retrieve → augment → generate flow
- The *response_mode* parameter controls the **response synthesis strategy**
For example: "*accumulate*" mode provides parallel answers by generating answers from each node independently and then concatenates them